Response Letter

We thank the reviewers for their valuable comments. The paper has been carefully revised according to the metareview, and all of the requested changes were implemented. Below is our detailed response to the metareview and each reviewer.

Replies to Metareview

Metareview Comment 1: Correct the description of SEDAR's comment-before-mutation and restore-after-mutation mechanism. Clearly explain the actual methodological differences between MORPH and SEDAR, avoiding misleading characterizations of the latter.
Reply: We have revised the description of SEDAR and the motivating example in Sections I (Introduction), II (Background and Motivation), and VI (Related Work), and we have clarified the methodological differences between MORPH and SEDAR in Sections II (Background and Motivation) and VI (Related Work).

Description of SEDAR. SEDAR translates each statement with the source-side sub-schema referenced by that statement. During mutation, it temporarily comments out fragments that the downstream fuzzer cannot parse and restores them before execution. However, the commented-out fragments are not mutated, and a fragment with a translation error still fails on the target DBMS after restoration. In addition, later statements are translated without the target-side schema and types produced by the translation of preceding statements. Figure 1 below shows both limitations on a test case transferred by SEDAR from POSTGRESQL to DuckDB. SEDAR translates `MONEY` in S1 to `DECIMAL(18,2)`, but S2′ keeps the money literal `'$12.50'`, which DuckDB cannot convert to `DECIMAL`. In S3′, the `to_date` fragment is commented out during mutation and restored before execution, and still fails because DuckDB does not provide `to_date`.

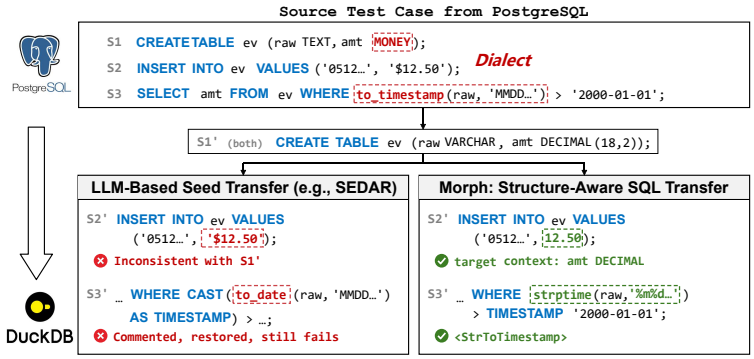


Figure 1: Motivating example of running SEDAR on a POSTGRESQL seed (Left): S2′ is inconsistent with the target-side type in S1′, and the incompatible fragment in S3′ is commented out during mutation and restored before execution, leaving the translation error unrepaired. MORPH abstracts dialect-specific logic via AQT and instantiates it with the target-side context in the target dialect (Right).

Differences in MORPH. MORPH addresses both limitations before mutation. First, it repairs incompatible fragments using parser and execution feedback. Thus, fragments that SEDAR comments out during mutation are translated into target-compatible SQL and remain available for mutation. Second, the AQT records forward read-after-write dependencies and the cumulative schema context of the source test case, and MORPH updates the target-side schema and type context af-

ter each translated statement, so later statements are instantiated against the translated target-side state. In Figure 1, MORPH uses the type of `amt` in S1′ to instantiate the literal in S2′ as `12.50`, and instantiates <`StrToTimestamp`> in S3′ with `strptime`, so all statements execute on DuckDB.

Experimental results. The evaluation in Section IV (Evaluation) shows the effect of these differences. Compared with SEDAR, MORPH achieves 5%–19% higher syntactic correctness and 100%–272% higher execution validity across the four target DBMSs (Table IV), and retains 22%–113% more functions, up to 62% more data types, and 23%–146% more keywords in the transferred seeds (Table V), as summarized in the table below. In 24-hour fuzzing campaigns, MORPH covers 14% more branches (Figure 7) and detects 16 more bugs than SEDAR.

Comparison of SEDAR and MORPH on transferred seeds (from Tables IV and V).

DBMS	Syn.		Exec.		F		DT		K	
	SEDAR	MORPH	SEDAR	MORPH	SEDAR	MORPH	SEDAR	MORPH	SEDAR	MORPH
MonetDB	0.781	**0.929**	0.388	**0.776**	132	**161**	29	29	244	**370**
Virtuoso	0.562	**0.670**	0.169	**0.629**	52	**87**	17	**24**	136	**335**
DuckDB	0.895	**0.939**	0.365	**0.748**	214	**456**	39	**63**	340	**476**
ClickHouse	0.821	**0.895**	0.315	**0.694**	227	**332**	61	**64**	152	**187**

Syn./Exec.: syntactic correctness/execution validity; F/DT/K: valid functions/data types/keywords in the transferred seeds.

Metareview Comment 2: Moderate the novelty claims and better distinguish MORPH from existing SQL translation approaches, particularly SEDAR, RISE, QTRAN, and CrackSQL. Explain more concretely what AQT and its associated mechanisms contribute beyond existing techniques, especially in the context of DBMS fuzzing.

Reply: We have revised Sections I (Introduction), II (Background and Motivation), III (Approach), and VI (Related Work) to clarify how MORPH differs from existing approaches, including query migration approaches (CrackSQL and RISE), oracle transfer approaches (QTRAN), and fuzzing seed transfer approaches (SEDAR). Below, we compare these approaches with MORPH in terms of their goals, challenges, methods, and experimental results.

Different goals. CrackSQL and RISE target query migration: they translate one self-contained query or procedural artifact at a time and aim to produce an equivalent target query. QTRAN targets oracle transfer: it translates a query and regenerates its target-side mutant so that the metamorphic relation is preserved for logic-bug detection. SEDAR and MORPH share the goal of transferring existing test cases into fuzzing seeds. For this goal, a transferred seed must execute on the target DBMS, retain testing-relevant SQL structures, and remain parsable and mutable by the downstream fuzzer.

Different challenges. For CrackSQL and RISE, the main challenge is to map the dialect-specific functions, types, and syntax of a single query or procedural artifact to an equivalent target form. For QTRAN, the main challenge is to keep the metamorphic relation between a query and its mutant valid after translation. For SEDAR, the main challenge is to make transferred test cases executable on the target DBMS and parsable by the downstream fuzzer, which it addresses by translating each statement with its referenced source-side sub-schema and temporarily commenting out incompatible fragments during mutation. For MORPH, the main challenge is that SQL logic is tightly coupled with dialect-specific syntax, which requires separating syntactic differences while preserving the structure, cross-statement dependencies, and semantic richness of a test case.

Different methods. CrackSQL combines functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. RISE reduces a complex query or procedural artifact to a simpler form, learns the local dialect rewrite from the reduced

form, and applies the rewrite back to the original artifact. QTRAN uses dialect knowledge and retrieval to translate a query and regenerate its target-side mutant. SEDAR uses an LLM to translate each statement under its referenced source-side sub-schema, and temporarily comments out parser-incompatible fragments during mutation and restores them before execution. In contrast, MORPH represents each source test case as an AQT that records normalized statement structures, cross-statement dependencies, and schema context, and instantiates it under the target dialect while propagating the translated target-side context across statements. We have updated Table I and Sections II (Background and Motivation) and VI (Related Work) to reflect these differences.

Different results. As a result, across the four target DBMSs, MORPH achieves 7%–528% higher branch coverage and detects 5–22 more bugs than the six evaluated fuzzing tools in 24-hour campaigns, including 14% higher coverage and 16 more bugs than SEDAR. It also improves execution validity by 100%–450% over the translation baselines.

Metareview Comment 3: Clarify how MORPH differs from and complements SmartFuzz in seed construction. The additional comparison is useful but limited in its experimental scope. The revision should acknowledge these limitations and avoid overstating comparative advantages.
Reply: We have clarified how MORPH differs from SmartFuzz in Section VI (Related Work), and extended the comparison in Section IV (Evaluation). In terms of goal, SmartFuzz generates new DBMS-specific seeds, whereas MORPH transfers existing test cases across DBMSs. In terms of methodology, SmartFuzz extracts SQL features from the official documentation of the target DBMS and bug-relevant features from historical crash-triggering inputs, and prompts an LLM with combinations of these features to generate seeds. In contrast, MORPH uses cross-statement dependencies and cumulative schema and type context to translate and repair existing test cases.

To compare them under the same setting, we use the seeds released by SmartFuzz and run them with GRIFFIN on all four target DBMSs, with the same DBMS versions, resource allocation, campaign duration, and coverage measurement as the other tools in RQ2. As shown in Figure 7 and Table III below, MORPH covers 7% more branches and detects 5 more bugs than SmartFuzz. On MonetDB, DuckDB, and ClickHouse, MORPH achieves higher coverage than SmartFuzz throughout the campaign. This advantage stems from the smaller dialect gaps. On Virtuoso, whose dialect differs most from the source DBMSs (the lowest execution validity of MORPH in Table IV), MORPH grows coverage more slowly at the beginning of the campaign, since it transfers seeds across a larger dialect gap, whereas SmartFuzz generates seeds directly for the target DBMS. MORPH nevertheless reaches coverage close to that of SmartFuzz by the end of the 24-hour campaign (40,183 vs. 40,773 branches), which shows the effectiveness of the transfer.

TABLE III: Number of bugs detected by different tools (24h).

DBMS	SQLancer	SQLRight	Squirrel	GRIFFIN	SEDAR	SmartFuzz	MORPH
MonetDB	-	0	0	0	3	11	**14**
Virtuoso	-	0	0	0	2	**3**	**3**
DuckDB	0	0	0	0	1	2	**3**
ClickHouse	0	0	-	0	0	1	**2**
Total	0	0	0	0	6	17	22
Increment	22↑	22↑	22↑	22↑	16↑	5↑	-

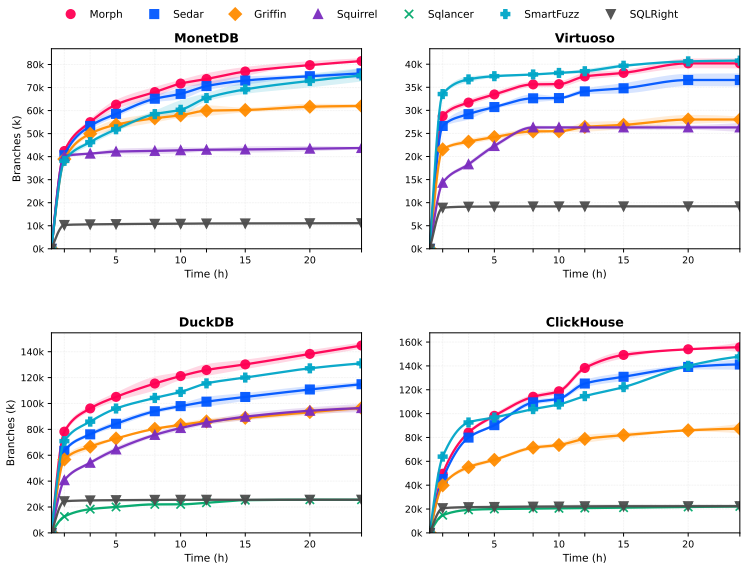


Figure 7: Coverage growth over 24-hour fuzzing campaigns. Displayed are the means and the 95% confidence intervals of five runs.

To examine their complementarity, we further run GRIFFIN with the union of the SmartFuzz and MORPH seeds. The combination covers 413,021 branches in total, 4.6% more than SmartFuzz alone and 2.2% fewer than MORPH alone, and detects 35 bugs, i.e., 13 and 18 more bugs than MORPH alone and SmartFuzz alone, respectively.

Metareview Comment 4: Clarify the provenance of the SEDAR binary used in the experiments, particularly given conflicting reports about its public availability. Provide sufficient information about the artifact and configuration to support reproducibility.
Reply: We have clarified the provenance of the SEDAR binary in Section IV (Evaluation), stating that the SEDAR binary used in our experiments was obtained from the authors of SEDAR. This binary runs the complete SEDAR pipeline, including schema capture, end-to-end LLM translation, and mutability refinement. To support reproducibility, we have included the SEDAR binary, its experimental configuration, and all SEDAR-generated seeds used in our experiments in our artifact.

Replies to Reviewer A

Question 1.1: Can the authors evaluate source-to-target result or state equivalence?
Reply: We have added a source-to-target result and state equivalence study in Section IV (Evaluation). On 100 randomly sampled MySQL test cases transferred to DuckDB, MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 14.0%/56.0% for SQLGlot, 34.0%/65.0% for RISE, and 38.0%/69.0% for SEDAR. We have also renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV (Evaluation) to distinguish runtime executability from result or state equivalence.

Source-to-target equivalence on 100 MySQL-to-DuckDB test cases.

Method	Result Eq. (%)	State Eq. (%)
SQLGlot	14.0	56.0
RISE	34.0	65.0
SEDAR	38.0	69.0
MORPH	**52.0**	**74.0**

Question 1.2: What independent benefit does the dependency graph provide beyond context propagation?
Reply: The dependency graph guides context selection during instantiation and repair. Without it, MORPH lacks explicit links for selecting dependency-relevant statements and object-state summaries for prompting. For example, in `ALTER TABLE t ADD COLUMN b INT; SELECT b FROM t;`, the edge selects the schema update for the translation and repair of the `SELECT`. Summaries provide the content, while edges guide the selection.

To isolate this benefit, we have added an ablation study in Section IV (Evaluation). We compare four context-selection modes under the same LLM, knowledge base, source corpus, repair budget, and statement order: (1) MORPH, which selects predecessors through the dependency graph; (2) MORPH$_{win}$, which replaces dependency-based selection with the five most recent successfully translated statements while keeping the same object summaries, so that it differs from MORPH only in whether the dependency graph is used; (3) MORPH$_{all}$, which provides all preceding successfully translated statements together with the complete source-side and target-side context; and (4) MORPH$_{w/o\text{-}ctx}$, which selects the same predecessors as MORPH but provides only their source-side statements, without their translated target-side statements or target-side object-state summaries. We measure execution validity at the statement level, the number of predecessors provided per statement, the proportion of predecessors that are dependency-relevant (i.e., connected to the current statement by a dependency edge), and input tokens per statement.

TABLE VI: Dependency-graph ablation across all source-target pairs.

Variant	Exec.	Preds	Rel. (%)	Tokens
MORPH$_{full}$	**0.716**	0.47	100.0	509.6
MORPH$_{win}$	0.599	1.19	22.6	578.4
MORPH$_{all}$	0.643	27.95	4.2	1,842.3
MORPH$_{w/o\text{-}ctx}$	0.523	0.49	100.0	488.5

Preds: predecessors per statement; Rel.: dependency-relevant predecessors;
Tokens: input tokens per statement.

MORPH achieves higher execution validity than MORPH$_{win}$ and MORPH$_{all}$ (0.716 vs. 0.599 and 0.643), while reducing input tokens per statement by 11.9% and 72.3%, respectively. In

MORPH$_{win}$ and MORPH$_{all}$, only 22.6% and 4.2% of the provided predecessors are dependency-relevant. In addition, 15.4% of statements select dependency predecessors outside the five most recent statements, which a recency-based window cannot capture. The context of MORPH$_{all}$ grows with test-case length: on test cases with more than 20 statements, its input tokens per statement are 4.98× those of MORPH. These results show that the independent benefit of the dependency graph lies in selecting a compact, dependency-relevant context whose size does not grow with test-case length, while achieving higher execution validity than larger contexts.

Question 1.3: How were CrackSQL, RISE, and SEDAR adapted and configured for the four target DBMSs, and how did the authors ensure a fair comparison?
Reply: We have added the configuration of the translation baselines in Section IV (Evaluation). Each baseline follows its original translation workflow. SQLGlot and jOOQ use their built-in dialect support. CrackSQL and RISE use the official SQL documentation of each target DBMS as dialect knowledge, i.e., the MonetDB SQL Reference, the Virtuoso Documentation, the DuckDB SQL Documentation, and the ClickHouse SQL Reference. Since CrackSQL and RISE are designed to translate one query or artifact at a time, we translate the statements of each test case individually and reassemble them in their original order, without schema context. SEDAR requires no additional dialect knowledge; we run the original SEDAR binary obtained from its authors, which translates complete test cases using the extracted source schemas as context.

To ensure a fair comparison, all methods transfer the same collected source test files, and no method receives additional manually written translation rules. The transferred seeds of all methods are evaluated on the same target DBMS versions (MonetDB v11.51, Virtuoso v7.2, DuckDB v1.4.3, and ClickHouse v24.8 LTS) with the same measurement of syntactic correctness and execution validity. In fuzzing, all compared methods run for 24 hours with the same DBMS versions, resource allocation, and coverage measurement, and the results are averaged over five independent runs. The prompts and configurations of all baselines are included in our artifact.

Comment 1.1: MORPH should therefore not be positioned as the first whole-test-case transfer framework or the first system to adapt transferred seeds to downstream fuzzers.
Reply: We have revised Sections I (Introduction), II (Background and Motivation), III (Approach), and VI (Related Work) to clarify how MORPH differs from existing approaches to whole-test-case transfer and adaptation of transferred seeds to downstream fuzzers (i.e., SEDAR). Details can be found in the reply to **Metareview Comment ??** and the reply to **Metareview Comment ??**.

Comment 1.2: These works show that structural decomposition, localized translation, and dialect-aware retrieval are not individually new. However, none of them models an independently executed stateful fuzzing test sequence, maintains target-side context across its statements, or integrates the result with coverage-guided seed mutation in the manner proposed by MORPH.
Reply: We have adopted this suggestion and further emphasized the contributions of MORPH in Section VI (Related Work). MORPH maintains evolving target-side context across an ordered test sequence and produces seeds that remain parsable and mutable by downstream fuzzers.

Comment 1.3: The related-work discussion contains several details that should be corrected. In particular, the paper states that CrackSQL employs genetic algorithms to mine mutation patterns, but the cited work instead uses functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. In addition, describing RISE as strictly processing a single SQL statement is too narrow because it also evaluates complete stored procedures containing control flow and multiple DML statements. A more accurate distinction is that RISE processes one self-contained query or procedural artifact at a time rather than a test file containing an ordered sequence of independently executed statements.
Reply: We have revised the description of CrackSQL in Section VI (Related Work): CrackSQL combines functionality-based query processing, cross-dialect syntax embedding with contrastive learning, and local-to-global translation. We have also clarified that RISE processes one self-contained query or procedural artifact at a time: it reduces the artifact to a simpler form, learns the local dialect rewrite from the reduced form, and applies the rewrite back to the original artifact.

Comment 1.4: It is therefore unclear what (G_{tc}) contributes independently beyond recording the dependencies and preserving an order that is already present. The reported ablation removes context propagation rather than the graph itself, so it establishes the value of context but not an independent benefit from (G_{tc}). The paper should explain whether the graph affects retrieval, prompting, repair, or any other step not apparent from the algorithms.
Reply: We have revised Section III (Approach) to make the role of the dependency graph explicit: it selects the dependency-relevant context used in the instantiation and repair of each statement. We have revised Algorithm 2 accordingly and added an ablation study in Section IV (Evaluation) that isolates this role from context propagation.

Role of the dependency graph. For each statement, the new SELECTCONTEXT step follows the dependency edges to select the preceding statements that write the objects it reads, and provides their source and translated statements in the prompts for instantiation and repair. This gives the LLM the schema changes that the statement depends on, even when they occur far back, without unrelated statements. Accordingly, the graph selects only dependency-relevant predecessors (100% for the dependency graph, 22.6% for the five-statement recency window, and 4.2% for the full context), captures the 15.4% of statements whose dependencies lie beyond the window, uses 11.9% fewer input tokens per statement than the five-statement recency window and 72.3% fewer than the full context, and achieves the highest execution validity (0.716 for the dependency graph, 0.599 for the five-statement recency window, and 0.643 for the full context). Details can be found in the reply to **Question ??**.

Revision of Algorithm 2. The main change is the new SELECTCONTEXT step (Line 4), which selects the dependency-relevant context $C_t^{(i)}$ for each statement, and INSTANTIATESTMT now uses $C_t^{(i)}$ instead of the full target context C_t. In addition, since forward read-after-write dependencies always point from earlier to later statements, the original statement order already satisfies them, and we have removed the stable topological sort. The revised Algorithm 2 is shown below, with removed parts in red and added parts in blue.

Algorithm 2: AQT Instantiation

Input : AQT $\mathcal{T} = \langle \mathbf{U}, \Delta, \mathscr{D}, \mathscr{G}_{tc}, \mathscr{H} \rangle$; target dialect D_t; Hybrid KB HKB; threshold δ
Output: Compatible seed sequence $\mathscr{Q}_{target}$

~~$StmtOrder \leftarrow$ StableTopologicalSort($\mathscr{G}_{tc}$, $\langle 1, \ldots, |\mathbf{U}| \rangle$);~~
$\mathscr{Q}_{target} \leftarrow [\,]$; $C_s \leftarrow \mathscr{D}$; $C_t \leftarrow \emptyset$;
for ~~i in $StmtOrder$~~ $i \leftarrow 1$ *to* $|\mathbf{U}|$ **do**
  $C_t^{(i)} \leftarrow$ SelectContext(C_t, $\mathscr{Q}_{target}$, Δ, i) ; // Dependency-relevant context
  $Q_i \leftarrow$ InstantiateStmt($\mathscr{U}_i, D_t, HKB, \delta, C_s$, ~~$C_t$~~ $C_t^{(i)}$);
  $\mathscr{Q}_{target}$.Append(Q_i);
  $C_t \leftarrow$ UpdateContext(C_t, Q_i) ; // Propagate context
IndexArtifact($(\mathscr{H}, D_t), \mathscr{Q}_{target}$);
return $\mathscr{Q}_{target}$;

Comment 1.5: The paper should also define the content and construction of the "object-state summaries" in (D) and clarify which forms of state are represented. The current formal definition explicitly supports forward RAW dependencies and cumulative schema/type context. Broader descriptions such as preserving all schema, data, and state dependencies should be narrowed unless additional dependency forms are implemented.
Reply: We have defined the object-state summaries in Section III (Approach). Each summary records whether a schema object exists, its table definition, column names and types, constraints, and the schema changes induced by preceding DDL statements, and is updated incrementally in statement order. We have also limited the descriptions in Sections III (Approach) and IV (Evaluation) to forward read-after-write dependencies and cumulative schema and type context.

Comment 1.6: I suggest renaming this metric to "Execution Validity" or "Runtime Validity" and narrowing statements concerning equivalent syntax, semantic-intent preservation, and high-fidelity transfer. A sampled source-to-target behavioral comparison would provide useful supplementary evidence.
Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper and clarified its definition in Section IV (Evaluation). We have also limited the description of instantiation in Section III (Approach) to keeping the generated queries valid in the target dialect. Section III also states that constant substitution and minimal commenting may change the behavior of transferred seeds. We have also added a sampled source-to-target result and state equivalence study, in which MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 38.0% and 69.0% for SEDAR. Details can be found in the reply to **Question ??**.

Comment 1.7: The paper should explain how each baseline was extended to these target dialects, including the dialect documents, knowledge bases, translation rules, LLMs, prompts, and manual effort used. It should also explain how a statement- or artifact-level translator was applied to a complete test sequence and what schema context, if any, was provided.
Reply: We have clarified the baseline configurations in Section IV (Evaluation). CrackSQL and RISE use official target-DBMS documentation and translate statements individually without schema context, while SEDAR translates complete test cases using extracted source schemas. Details can be found in the reply to **Question ??**.

Comment 1.8: The authors should confirm whether their reproduction includes the restoration step. If the motivating example is an observed failure in the reproduced system rather than the general SEDAR algorithm, this distinction should be stated explicitly.

Reply: We have clarified in Section IV (Evaluation) that our experiments use the SEDAR binary obtained from the authors of SEDAR, which runs its complete pipeline, including the restoration step of mutability refinement. We have revised the descriptions of SEDAR in Sections I (Introduction), II (Background and Motivation), and IV (Evaluation) accordingly. We have also revised the motivating example in Section II (Background and Motivation) and stated explicitly that it is a failure observed when running SEDAR on a POSTGRESQL seed, rather than a general property of the SEDAR algorithm. In this example, S2′ keeps the money literal `'$12.50'`, which DuckDB cannot convert to the `DECIMAL(18,2)` column created by S1′, and S3′ uses `to_date`, which DuckDB does not provide. SEDAR comments out the `to_date` fragment during mutation and restores it before execution, so the restored statement still fails on DuckDB, whereas MORPH instantiates the literal as `12.50` and the function with `strptime`, and all statements execute on DuckDB. Details can be found in the reply to **Metareview Comment ??**.

Comment 1.9: The paper reports average coverage over five runs, but Tables III and VII provide no variance, confidence intervals, or statistical tests. Such information is important for stochastic fuzzing experiments. It is also unclear whether the reported bug counts are unions across the five runs or results from a particular run.

Reply: We have reported the statistics of the coverage results in Section IV (Evaluation). (1) *Statistical tests*: the 8%–39% coverage gains from enabling the feedback-driven sanitizer (Table VII) and the 4.0%–27.2% coverage gains of MORPH's mutation handling over that of SEDAR are statistically significant (Mann–Whitney U test, $p < 0.05$). (2) *Variability*: all coverage results are averaged over five independent runs; for MORPH, the variance is 5.4×10^6, the standard deviation is 2,324, and the 95% confidence interval is $\pm 2,900$. (3) *Bug counts*: the bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification.

We have also revised Figure 7 to display the means and the 95% confidence intervals of the five runs, shown below, and stated in Section IV (Evaluation) how the coverage results and bug counts are aggregated.

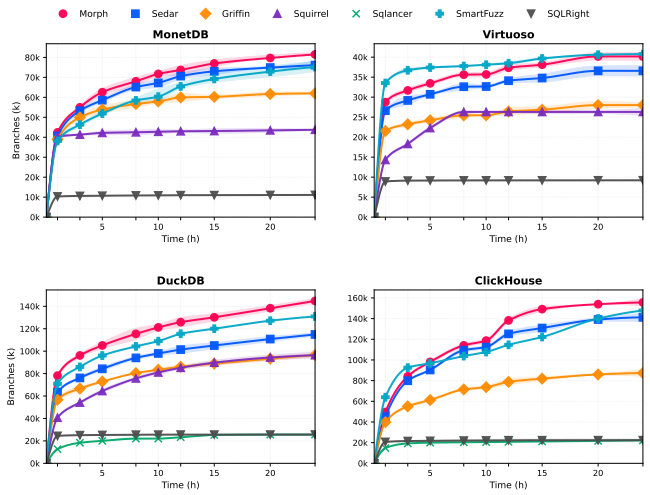


Figure 7: Coverage growth over 24-hour fuzzing campaigns. Displayed are the means and the 95% confidence intervals of five runs.

Comment 1.10: The full bug-finding campaign discovers 28 bugs, whereas the 24-hour comparison reports 22. The duration and computational budget of the full campaign should be reported. Similarly, Table VIII should specify the currency of the cost, the unit of time, and the total offline cost of processing all test files.

Reply: We have reported in Section IV (Evaluation) that the full campaign in RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 compares fuzzing tools in five independent 24-hour runs and reports the union of unique bugs across these runs. We have also added the units to the column headers of Table VIII, i.e., Cost/TS ($) and Time/TS (s), and we have reported in Section V (Discussion) that processing all test files with Gemini 2.5-Flash-Lite costs $96 in total.

Comment 1.11: Finally, the component ablation is performed only for PostgreSQL-to-DuckDB transfer. Extending at least the most important ablation, particularly context propagation, to additional source-target pairs would strengthen the generality of the findings.

Reply: We have extended the entire component ablation in RQ4 from the POSTGRESQL-to-DuckDB transfer to all source-target pairs used in RQ3, and have updated the corresponding figure and tables in Section IV (Evaluation). Across these pairs, MORPH$_{abs}$ improves syntactic correctness from 43.1% to 73.6%, MORPH$_{rag}$ improves execution validity from 35.2% to 61.9%, and MORPH$_{full}$ achieves the highest syntactic correctness (87.1%) and execution validity (71.6%) (Figure 8). In particular, across these pairs, providing only the source-side statements of the same predecessors, without their translated statements or target-side object-state summaries, reduces execution validity from 0.716 to 0.523 (Table VI). The same table also reports the dependency-graph ablation. Details can be found in the reply to **Question ??**.

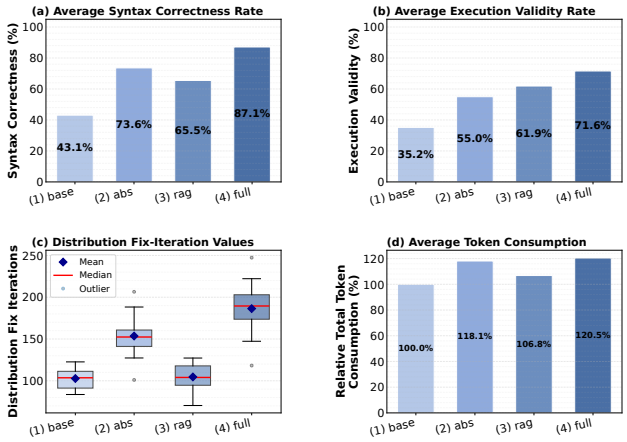


Figure 8: Ablation analysis of MORPH components across all source-target pairs.

Comment 1.12: The paper reports the target DBMS versions, corpus size, primary model, temperature, hardware environment, and an anonymous artifact.

For full reproducibility, the paper should additionally document the baseline adaptations, source-target test allocation, repeated-run aggregation procedure, full campaign duration, and validation procedure for automatically generated translation rules. Table IX reports that 136 rules required correction.

Reply: We have documented these details in the revised paper. (1) *Baseline adaptations:* Section IV (Evaluation) describes the dialect knowledge used by each baseline and how statement-level translators are applied to complete test cases (the reply to **Comment ??**). (2) *Source-target*

test allocation: the experimental setup reports that all methods transfer the same collected test files and use the same number of initial seeds for each target DBMS, i.e., 5,879, 4,551, 5,850, and 5,990 seeds for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively. (3) *Repeated-run aggregation:* coverage is averaged over five independent runs, Figure 7 displays the means and the 95% confidence intervals, and bug counts are the union of unique bugs across the five runs after stack-trace deduplication and manual verification (the reply to **Comment ??**). (4) *Full campaign duration:* RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 consists of five independent 24-hour runs (the reply to **Comment ??**). (5) *Validation of generated rules:* Section V (Discussion) now describes that the generated rules are applied to translated test cases and checked for errors, and that rules producing incorrect translations are manually inspected and corrected, which accounts for the 136 corrected rules in Table IX.

Replies to Reviewer B

Question 2.1: Could the authors clarify whether the reported coverage/bug-finding improvements primarily stem from AQT-based translation or from a larger/richer effective seed corpus? In a controlled experiment where all baselines use exactly the same number and source of initial seeds, how much of the 14% coverage gain over SEDAR remains attributable to AQT?
Reply: *Initial Seed Settings.* In the comparison in Section IV (Evaluation), we use the same number of initial seeds for MORPH and SEDAR, i.e., 5,879, 4,551, 5,850, and 5,990 seeds for MonetDB, Virtuoso, DuckDB, and ClickHouse, respectively. As shown in the table below, MORPH covers 14.5% more branches and detects 16 more bugs than SEDAR, indicating that the observed improvement comes from AQT rather than the number of initial seeds.

Branch coverage of SEDAR and MORPH with the full seed corpora (24h).

DBMS	SEDAR	MORPH	Improvement
MonetDB	76,142	**81,486**	7.0%
Virtuoso	36,583	**40,183**	9.8%
DuckDB	114,876	**144,822**	26.1%
ClickHouse	141,215	**155,639**	10.2%
Total	368,816	**422,130**	14.5%

Seed-controlled experiment. We have also added a seed-controlled experiment in Section IV (Evaluation). With the same number of initial seeds, MORPH achieves 17.2%–92.4% higher coverage than SEDAR across the four target DBMSs. Since the seed count is controlled, this improvement comes from how MORPH transfers test cases: it represents each source test case as an AQT that records statement structures, cross-statement dependencies, and schema context, and instantiates the AQT under the target dialect while propagating the translated target-side context across statements. In this experiment, both methods transfer the same source test cases, and for each target DBMS, we randomly sample 100, 200, and 300 seeds from the seeds generated by each method and use them as the initial seeds of GRIFFIN. Each configuration runs for 24 hours and is repeated five times, with the same DBMS version, GRIFFIN configuration, and resource allocation. As shown in the table below, MORPH achieves 47.3%–65.2% higher coverage than SEDAR on DuckDB, 17.2%–32.4% on MonetDB, 49.7%–75.0% on Virtuoso, and 57.8%–92.4% on ClickHouse. In addition, SEDAR detects no bugs in these settings, whereas MORPH detects one

MonetDB bug with 200 and 300 seeds and one Virtuoso bug. These results show that although seed quantity affects coverage, MORPH outperforms SEDAR under the same seed size.

Branch coverage of SEDAR and MORPH with the same number of initial seeds.

DBMS	100 seeds		200 seeds		300 seeds	
	SEDAR	MORPH	SEDAR	MORPH	SEDAR	MORPH
DuckDB	49,745	**82,197**	58,938	**92,390**	68,499	**100,903**
MonetDB	20,406	**27,025**	28,173	**33,006**	29,948	**35,703**
Virtuoso	13,717	**24,002**	16,107	**26,670**	19,869	**29,745**
ClickHouse	35,967	**67,798**	41,478	**79,806**	55,852	**88,149**

Question 2.2: Regarding the characterization of SEDAR's comment-and-restore mechanism—if both MORPH and SEDAR ultimately feed compatible seeds to GRIFFIN, what is the exact difference in their post-mutation handling? Do seeds repaired by MORPH produce semantically equivalent mutated queries, whereas SEDAR's approach loses that equivalence? If not, what is the claimed advantage?

Reply: We have clarified the difference between the two mutation-handling strategies (i.e., pre-mutation repair in MORPH vs. post-mutation restoration in SEDAR) in Sections II (Background and Motivation) and IV (Evaluation), and we have added a source-to-target equivalence study and a controlled coverage comparison in Section IV (Evaluation).

Difference and advantage. MORPH performs pre-migration repair to make seeds parser-compatible, whereas SEDAR defers some compatibility handling until after mutation: it temporarily comments out fragments that the fuzzer cannot parse during mutation and restores them before execution. Neither strategy guarantees semantic equivalence after mutation. Nevertheless, MORPH achieves higher coverage because it resolves incompatibilities before mutation. It repairs incompatible fragments using parser and execution feedback, so the transferred seeds are accepted by both the fuzzer and the target DBMS. This has two benefits: (1) the fuzzer can mutate the repaired fragments, so mutation explores more SQL constructs; and (2) fragments with translation errors are repaired rather than restored as is (e.g., the `to_date` fragment in Figure 1, the reply to **Metareview Comment ??**), so fewer mutated test cases fail on the target DBMS and more of them reach deeper execution paths.

Experimental results. The transferred seeds of MORPH achieve 100%–272% higher execution validity than those of SEDAR across the four target DBMSs (Table IV). To isolate the effect of mutation handling, we have sampled the same 300 translated seeds for each target DBMS and applied the respective mutation handling of MORPH and SEDAR with GRIFFIN for 24 hours. MORPH's handling finds 27.2%, 15.5%, 4.0%, and 9.4% more branches than that of SEDAR on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively ($p < 0.05$). In addition, on 100 sampled MySQL-to-DuckDB test cases, MORPH achieves higher result and state equivalence than SEDAR (52.0%/74.0% vs. 38.0%/69.0%), as shown in the table below.

Source-to-target equivalence on 100 MySQL-to-DuckDB test cases.

Method	Result Eq. (%)	State Eq. (%)
SQLGlot	14.0	56.0
RISE	34.0	65.0
SEDAR	38.0	69.0
MORPH	**52.0**	**74.0**

Comment 2.1: Limited novelty. The paper positions AQT as a key contribution, but AQT largely resembles an existing SQL AST or intermediate representation augmented with abstract labels, read/write sets, dependency edges, and schema context. The authors do not clearly articulate what fundamentally new capability AQT provides over existing SQL representations (e.g., JOOQ's parse tree or SQLGlot's AST) or translation systems (CrackSQL, RISE, QTRAN). The distinction from these systems is insufficiently developed.

More importantly, the evaluation lacks compositional baselines such as "existing SQL translator + SEDAR/GRIFFIN" or "AST-based translation + GRIFFIN". Without such ablations, it is difficult to determine whether the contribution is a fundamentally new transfer technique or mainly an engineering integration of existing components. The authors should add such baselines or provide a clearer theoretical argument for why AQT enables capabilities that prior representations cannot achieve.

Reply: We have clarified in Sections II (Background and Motivation), III (Approach), and VI (Related Work) that AQT's novelty lies in *jointly modeling cross-statement dependencies and cumulative schema/type context for coordinated translation/repair*. Unlike statement-level representations (e.g., SQLGlot's AST), which primarily represent individual SQL statements, AQT models the evolving state of an entire test sequence; unlike query-level translation systems (e.g., CrackSQL/RISE/QTRAN), AQT uses preceding statements to guide the translation/repair of subsequent statements.

We have further paired different translators with the same downstream fuzzer (i.e., GRIFFIN) for 24 hours on DuckDB in Section IV (Evaluation). MORPH finds 33.6%, 16.7%, and 26.1% more branches and 3, 3, and 2 more bugs than SQLGlot+GRIFFIN, RISE+GRIFFIN, and SEDAR+GRIFFIN, respectively, demonstrating the effectiveness of MORPH's translation approach.

Compositional baselines on DuckDB (24h).

Method	Branch Coverage	Unique Bugs
SQLGlot + GRIFFIN	108,385	0
RISE + GRIFFIN	124,147	0
SEDAR + GRIFFIN	114,876	1
MORPH + GRIFFIN	144,822	3

Comment 2.2: Incomplete discussion of closely related work. SmartFuzz (OOPSLA 2026) also studies high-quality seed generation for DBMS fuzzing and is directly relevant to the problem addressed by this paper. However, the paper neither discusses it sufficiently nor compares against it experimentally. The authors should add a discussion of SmartFuzz in Section VI (Related Work), clarify the differences in approach, and ideally include it as a baseline in the evaluation if feasible.

Reply: We have clarified the differences in goal and methodology between SmartFuzz and MORPH in Section VI (Related Work).

Goal: SmartFuzz focuses on generating new DBMS-specific seeds, whereas MORPH focuses on transferring existing test cases across DBMSs.

Methodology: SmartFuzz extracts SQL features from the official documentation of the target DBMS and bug-relevant features from historical crash-triggering inputs, and prompts an LLM with combinations of these features to generate seeds. In contrast, MORPH uses cross-statement dependencies and cumulative schema and type context to translate and repair existing test cases.

We have also extended the comparison in Section IV (Evaluation). Across the four target DBMSs, MORPH covers 7% more branches in total and detects 5 more bugs than SmartFuzz. Details can be found in the reply to **Metareview Comment ??**.

Comment 2.3: Potentially inaccurate characterization of SEDAR. The paper characterizes SEDAR's comment-and-restore mechanism as "destructive sanitization" that "sacrifices semantic depth." However, SEDAR's mechanism is designed to temporarily hide target-specific constructs from downstream fuzzers' strict parsers during mutation and restore them afterward. This is a pragmatic compatibility strategy, not a destructive removal of query logic.

The authors should revise their characterization to accurately reflect SEDAR's design intent and clarify the actual difference: MORPH performs pre-migration repair to make seeds parser-compatible, whereas SEDAR defers some compatibility handling until after mutation. A fairer comparison would evaluate both approaches under the same downstream fuzzer (e.g., GRIFFIN) and measure the net effect of each strategy on mutation effectiveness.

Reply: We have revised the description of SEDAR in Sections I (Introduction), II (Background and Motivation), and VI (Related Work): SEDAR temporarily comments out parser-incompatible fragments during mutation and restores them before execution. We have also clarified the actual difference: MORPH repairs incompatibilities before mutation, whereas SEDAR defers compatibility handling until after mutation.

We have added a controlled comparison in Section IV (Evaluation) to evaluate the two mutation-handling strategies. For each of the four target DBMSs, we applied both strategies to the same 300 translated seeds and ran GRIFFIN for 24 hours in five independent repetitions. Compared with SEDAR's strategy, MORPH's strategy achieves 27.2%, 15.5%, 4.0%, and 9.4% higher branch coverage on DuckDB, MonetDB, Virtuoso, and ClickHouse, respectively.

Comment 2.4: Questionable classification in Table I. The classification of SQLGlot and jOOQ as entirely lacking "Test Case Migration" and "Cross-stmt Deps." support is overly absolute. In practice: SQLGlot can parse and transpile multi-statement SQL scripts. JOOQ's SQL Interpreter can maintain schema metadata across DDL and subsequent DML statements. Conversely, CrackSQL is marked as supporting DML, but this is not sufficiently validated by the paper or external references. The authors should: (a) clearly define what ✓, △, and × mean for each cell with specific criteria, (b) provide evidence for each classification (e.g., cite documentation or experimental verification), and (c) revise the table entries accordingly to avoid overclaiming.

Reply: We have clarified the criteria for the three symbols in Table I in Section II (Background and Motivation) and revised the entries accordingly. As evidence, we have cited the documentation of each tool, e.g., the SQLGlot API documentation for multi-statement transpilation and the jOOQ SQL interpreter documentation for schema metadata tracking. As marked below, SQLGlot's *Test Case Migration*, jOOQ's *Cross-stmt. Deps.*, SEDAR's *Cross-stmt. Deps.*, and CrackSQL's *DML* entries are now marked as △.

TABLE I: Capability comparison of existing SQL translation systems for test-case-level DBMS fuzzing seed transfer.

Tool	DDL	DML	DQL	Test Case Migration	Cross-stmt. Deps.
SQLGlot	✓	✓	✓	~~×~~ △	×
jOOQ	✓	✓	✓	×	~~×~~ △
RISE	△	△	✓	×	×
CrackSQL	△	~~✓~~ △	✓	×	×
QTRAN	△	△	✓	×	×
SEDAR	△	△	✓	✓	~~×~~ △
MORPH	✓	✓	✓	✓	✓

Cross-stmt. Deps. denotes explicit tracking of forward read-after-write dependencies and schema/type context across statements. ✓: supported and documented or evaluated; △: partially supported, i.e., supported only for some statements or settings, or through a separate component; ×: not supported.

Comment 2.5: Unclear source of the improvement. The reported coverage and bug-finding improvements may stem from a larger or more diverse effective seed corpus rather than the AQT-based translation technique itself. For a fair comparison with SEDAR, GRIFFIN, and SQUIRREL, all approaches should use exactly the same source seed corpus and produce comparable numbers of final seeds that pass the target parser.

The current experimental setup does not control for seed corpus size. The authors should: (a) report the number of valid seeds generated by each method before fuzzing, (b) normalize the corpus size across baselines, or (c) conduct an experiment where seed counts are equalized. This controlled setting is necessary to isolate the contribution of MORPH's SQL translation technique and demonstrate that improvements are not simply due to differences in seed quantity or diversity.
Reply: We have conducted an experiment where seed counts are equalized in Section IV (Evaluation). MORPH and SEDAR transfer the same source test cases, and all methods use the same number of initial seeds (100, 200, and 300). Under the same seed count, MORPH achieves TODO(x)%–TODO(y)%, TODO(x)%–TODO(y)%, and 17.2%–92.4% higher coverage than SQUIRREL, GRIFFIN, and SEDAR, respectively, across the four target DBMSs, as shown in the table below.

Branch coverage of different tools with the same number of initial seeds (24h).

DBMS	Seeds	SQUIRREL	GRIFFIN	SEDAR	MORPH
DuckDB	100	TODO()	TODO()	49,745	**82,197**
	200	TODO()	TODO()	58,938	**92,390**
	300	TODO()	TODO()	68,499	**100,903**
MonetDB	100	TODO()	TODO()	20,406	**27,025**
	200	TODO()	TODO()	28,173	**33,006**
	300	TODO()	TODO()	29,948	**35,703**
Virtuoso	100	TODO()	TODO()	13,717	**24,002**
	200	TODO()	TODO()	16,107	**26,670**
	300	TODO()	TODO()	19,869	**29,745**
ClickHouse	100	-	TODO()	35,967	**67,798**
	200	-	TODO()	41,478	**79,806**
	300	-	TODO()	55,852	**88,149**

-: SQUIRREL does not support ClickHouse.

Comment 2.6: Translation overhead is unclear. Table VIII reports "Cost/TS" and "Time/TS" but does not specify the unit of measurement (e.g., USD per test suite? seconds per test suite?) or the size of the test suite being translated. Additionally, the paper does not compare MORPH's end-to-end translation time and LLM cost with previous methods such as SEDAR.

The authors should: (a) clearly define the units for cost and time, (b) report the average number of statements per test suite, (c) report per-seed translation time and cost, and (d) compare these

overhead metrics against SEDAR and other baselines to demonstrate the practical feasibility of MORPH in real-world deployment.

Reply: We have clarified the translation overhead of MORPH in Section V (Discussion). (a) We have added the units to the column headers of Table VIII, i.e., Cost/TS ($) and Time/TS (s). (b) We have reported that each test suite contains 65.1 statements on average. (c) Across the four target DBMSs, MORPH takes 219 seconds and $0.004 per seed on average, and processing all test files costs $96 in total. (d) Under the same setting, SEDAR takes 146 seconds and $0.006 per seed, with a total cost of $144. MORPH costs less than SEDAR but takes longer per seed, since it performs additional instantiation and repair iterations. Because seed transfer is a one-time offline step and the seeds can be reused across fuzzing campaigns, this overhead is acceptable in practice.

TABLE VIII: LLM comparison. Syn.: syntactic correctness; Exec.: execution validity; TS ~~denotes~~ : Test Suite.

Model	Syn.	~~Sem.~~ Exec.	Cost/TS ($)	Time/TS (s)
Gemini 2.5-Flash-Lite	0.929	0.776	0.003	146.80
GPT-3.5-Turbo	0.890	0.533	0.015	208.66
DeepSeek-V3	0.986	0.699	0.003	231.11
Qwen3.5-27B*	0.971	0.852	0.119	1395.6

* Qwen3.5-27B with deep thinking mode.

Comment 2.7: Bug evidence is insufficient. The bug links are inaccessible, and the triggering test cases are not publicly available. The artifact includes complete and executable source code.

Reply: We have added the triggering test cases of all detected bugs to our anonymous artifact for bug verification and reproduction. The links to the bug reports will be released after acceptance to preserve double-blind review.

Replies to Reviewer C

Comment 3.1: The "semantic correctness" metric means only "executes without runtime errors," not result equivalence; a seed that runs but computes something different still counts. Using "+130-450%" as a headline overstates what was measured.

Reply: We have renamed "Semantic Correctness" to "Execution Validity" throughout the paper, including the abstract, the introduction, and Table IV. The headline improvements (e.g., 130% over SEDAR and 100%–450% over the baselines) are therefore reported as gains in execution validity, and the improvement over each baseline is computed only on the target DBMSs supported by both methods. We have also clarified in Section IV (Evaluation) that execution validity measures executability on the target DBMS and does not imply semantic equivalence. In a sampled equivalence study on 100 MySQL-to-DuckDB test cases, MORPH achieves 52.0% result equivalence and 74.0% state equivalence, compared with 38.0% and 69.0% for SEDAR. The revised Table IV is shown below.

TABLE IV: ~~Syntactic and semantic correctness~~ Syntactic correctness and execution validity of transferred seeds.

Method	MonetDB		Virtuoso		DuckDB		ClickHouse	
	Syn.	~~Sem.~~ Exec.	Syn.	~~Sem.~~ Exec.	Syn.	~~Sem.~~ Exec.	Syn.	~~Sem.~~ Exec.
No translation	0.589	0.236	0.324	0.097	0.674	0.284	0.613	0.182
SQLGlot	-	-	-	-	0.738	0.186	0.665	0.193
jOOQ	-	-	-	-	0.652	0.136	0.554	0.175
CrackSQL	-	-	-	-	0.740	0.211	0.717	0.264
RISE	0.830	0.266	0.545	0.178	0.857	0.314	0.764	0.278
SEDAR	0.781	0.388	0.562	0.169	0.895	0.365	0.821	0.315
MORPH	**0.929**	**0.776**	**0.670**	**0.629**	**0.939**	**0.748**	**0.895**	**0.694**
vs. SEDAR	+19%	+100%	+19%	+272%	+5%	+105%	+9%	+120%

Syn./~~Sem.~~ Exec.: ~~syntactic/semantic correctness~~ syntactic correctness/execution validity; "-": unsupported dialects.

Comment 3.2: RQ2 baselines are somewhat hard to believe: zero bugs in 24 hours for SQUIRREL, GRIFFIN, and SQLancer across all four DBMSs. The baseline configuration is undocumented, several cells are "-", for unsupported targets, and the "528% over SQLancer" headline is computed over only the two DBMSs that SQLancer supports.

Reply: We have documented the configuration of the RQ2 baselines in Section IV (Evaluation). For SQLANCER, SQUIRREL, and GRIFFIN, we use the default configurations, seed corpora, and command lines from their official repositories, and we have included them in our artifact. The zero-bug results reflect that previously reported bugs were fixed in the newer DBMS versions, with no new bugs found. In addition, the 528% improvement over SQLANCER is computed only on DuckDB and ClickHouse, the two target DBMSs it supports, as stated in Section IV (Evaluation).

Comment 3.3: Numeric inconsistency: the text claims RISE transfer improved semantic correctness "from 0.314 to 0.776," but Table V's own row reads RISE 0.314 / MORPH 0.748. One of these is wrong.

Reply: We have revised the text in Section IV (Evaluation) to state that, in the SQLite-to-DuckDB setting, MORPH improves execution validity from 0.314 (RISE) to 0.748, consistent with Table IV. We have also checked the other numbers reported in Section IV (Evaluation) against Table IV.

Comment 3.4: Missing related work: SQLRight (USENIX Sec 2022) absent from RQ2; sqllogictest is never discussed; classic seed-selection literature ignored.
Reply: We have added SQLRight as a baseline in RQ2. Since SQLRight officially supports only SQLite, PostgreSQL, and MySQL, we have adapted it to the four target DBMSs while keeping its AFL-based queue scheduling, AST-based mutator, and oracle framework, and we have included the adaptation code in our artifact. Across the four target DBMSs, MORPH covers 518% more branches than SQLRight and detects 22 more bugs. Details can be found in the reply to **Metareview Comment ??**.

We have also extended the related work in Section VI (Related Work) with SQLRight, sqllogictest, and seed-selection techniques. SQLRight performs coverage-guided DBMS fuzzing for logic bugs, whereas MORPH constructs initial seeds for DBMS fuzzers. sqllogictest validates query results across database engines with a shared test format, whereas MORPH transfers test cases into seeds for coverage-guided fuzzing. Seed-selection and scheduling techniques, such as corpus minimization and power schedules (e.g., AFLFast), choose or prioritize seeds within an existing corpus, whereas MORPH constructs the initial corpus for target DBMSs that lack one; the two directions are therefore orthogonal and can be combined.

Comment 3.5: What's novel here is the conjunction of approaches: AST abstraction + RAG + parser-feedback repair, which are established ideas assembled around the AQT.
Reply: We have clarified in Section VI (Related Work) that MORPH adopts these techniques but focuses on maintaining evolving target-side context across an ordered test sequence through the AQT. Specifically, the AQT records forward read-after-write dependencies and cumulative schema context, which MORPH uses to select dependency-relevant context and to propagate the translated target-side state across statements during instantiation and repair. We have also added an ablation study in Section IV (Evaluation) that isolates the contributions of the dependency graph and context propagation. As shown in Table VI below, dependency-based context selection achieves higher execution validity than a recency-based window and the full context (0.716 vs. 0.599 and 0.643) while using 11.9% and 72.3% fewer input tokens per statement, and providing only the source-side statements of the same predecessors reduces execution validity to 0.523. Details can be found in the reply to **Metareview Comment ??**.

TABLE VI: Dependency-graph ablation across all source-target pairs.

Variant	Exec.	Preds	Rel. (%)	Tokens
MORPH$_{full}$	**0.716**	0.47	100.0	509.6
MORPH$_{win}$	0.599	1.19	22.6	578.4
MORPH$_{all}$	0.643	27.95	4.2	1,842.3
MORPH$_{w/o\text{-}ctx}$	0.523	0.49	100.0	488.5

Preds: predecessors per statement; Rel.: dependency-relevant predecessors;
Tokens: input tokens per statement.

Comment 3.6: RQ1 (28 bugs, unspecified "full campaign") vs RQ2 (22 bugs in 24h) relationship and campaign duration are unclear.
Reply: The full campaign in RQ1 runs MORPH for seven days on each target DBMS, whereas RQ2 compares fuzzing tools in five independent 24-hour runs and reports the union of unique bugs across these runs.

Comment 3.7: I would recommed tightening the validity claims: add a small differential check on transferred seeds (do they return equivalent results on a shared dialect subset?) to back the "semantic correctness" framing, and lean on coverage/bug counts rather than feature-richness as the primary evidence. For RQ2, publish baseline command lines and seed corpora in the artifact; the all-zero bug rows need more transparency, and SQLRight belongs in the comparison.
Reply: We have revised the paper following these suggestions. (1) *Differential check:* we have renamed "Semantic Correctness" to "Execution Validity" and added a sampled source-to-target result and state equivalence study (52.0%/74.0% for MORPH vs. 38.0%/69.0% for SEDAR; the reply to **Comment ??**). (2) *Primary evidence:* our main claims rely on branch coverage and bug counts in RQ1 and RQ2, and the feature richness in RQ3 is presented as supplementary evidence of the testing value of transferred seeds. In 24-hour campaigns, MORPH achieves 7%–528% higher branch coverage and detects 5–22 more bugs than the six evaluated fuzzing tools (Figure 7 and Table III). (3) *Baseline transparency:* we have documented the configurations, seed corpora, and command lines of the RQ2 baselines (the defaults from their official repositories), included them in our artifact, and explained the zero-bug results (the reply to **Comment ??**). (4) *SQLRight:* we have added SQLRight to the RQ2 comparison; across the four target DBMSs, MORPH covers 518% more branches and detects 22 more bugs than SQLRight (the reply to **Comment ??**).